

Problem 2 :

Given a string *s*, composed of different combinations of '(', ')', '{', '}', '[', ']', Determine whether the Expression is balanced or not. An expression is balanced if: 1. Each opening bracket has a corresponding closing bracket of the same type. 2. Opening brackets must be closed in the correct order.

Concept: Stack

Examples :

Input: *s* = "[{}()]"
Output: true
Explanation: All the brackets are well-formed.

Input: *s* = "[()()]"
Output: true
Explanation: All the brackets are well-formed.

Input: *s* = "[]"
Output: false
Explanation: The expression is not balanced as there is a missing ')' at the end.

Input: *s* = "([{}])"
Output: false
Explanation: The expression is not balanced as there is a closing ']' before the closing ']'.

Code:

```
#include <stdio.h>
#include <string.h>
#define MAX 1000000
char stack[MAX];
int top = -1;
// Push function
void push(char c) {
    stack[++top] = c;
}
// Pop function
char pop() {
    if (top == -1)
        return '\0';
    return stack[top--];
}
// Check if matching pair
int isMatchingPair(char open, char close) {
    if (open == '(' && close == ')') return 1;
    if (open == '{' && close == '}') return 1;
    if (open == '[' && close == ']') return 1;
    return 0;
}
```

```

}

int isBalanced(char *s) {
    top = -1; // Reset stack
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] == '(' || s[i] == '{' || s[i] == '[') {
            push(s[i]);
        }
        else if (s[i] == ')' || s[i] == '}' || s[i] == ']') {
            if (top == -1)
                return 0;
            char topElement = pop();
            if (!isMatchingPair(topElement, s[i]))
                return 0;
        }
    }
    return (top == -1);
}

int main() {
    printf("Name : Kartika Patil\n");
    printf("PRN : 25070521219\n");
    printf("Section : D-2\n");
    printf("Value Added Problem : 2\n");
    printf("-----\n");
    char s[MAX];
    printf("Enter expression: ");
    fgets(s, MAX, stdin);
    if (isBalanced(s))
        printf("true\n");
    else
        printf("false\n");
    return 0;
}

```

Output:

The screenshot shows a C++ IDE with two panels. The left panel displays the source code for a program that checks if an expression is balanced. The right panel shows the output of the program, which includes the user's input and the program's response.

```

main.c
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 1000000
5
6 char stack[MAX];
7 int top = -1;
8
9 // Push function
10 void push(char c) {
11     stack[++top] = c;
12 }
13
14 // Pop function
15 char pop() {
16     if (top == -1)
17         return '\0';
18     return stack[top--];
19 }
20
21 // Check if matching pair
22 int isMatchingPair(char open, char close) {
23     if (open == '(' && close == ')') return 1;
24     if (open == '{' && close == '}') return 1;
25     if (open == '[' && close == ']') return 1;
26     return 0;
27 }

```

Output

```

Name : Kartika Patil
PRN : 25070521219
Section : D-2
Value Added Problem : 2
-----
Enter expression: {[()]}
true
=== Code Execution Successful ===

```